

Writing Text Data To Files In Java

Generated Aug 3, 2026 8:25 AM

What We'll Learn

By the end of this lesson, students can:

- Explain why programs save information to files.
- Create a text file using Java.
- Write text data into a file using `FileWriter`.
- Properly close a file after writing data.
- Handle possible file-writing errors using `try` and `catch`.

Before We Start

Think about the following situation:

You created a program that asks for a student's name and score. The program displays the information on the screen.

What happens to the information after the program closes?

How can we keep the information so that it can be opened and viewed later?

Let's Understand

Programs normally store values in memory while they are running.

When the program closes, those values are usually lost.

To keep information permanently, a Java program can write the data into a file.

A text file stores information using readable characters. Text files commonly use the .txt file extension.

Examples include:

- Student records
- Customer receipts
- Attendance lists
- Activity results
- Program reports

Java provides the `FileWriter` class for writing text into a file.

To use `FileWriter`, we import it at the beginning of the program:

```
import java.io.FileWriter;
```

We also use `IOException` because an error may occur while creating or writing to a file.

```
import java.io.IOException;
```

A basic file-writing program follows these steps:

1. Create or open a text file.
2. Write text into the file.
3. Close the file.
4. Handle possible errors.

The basic structure is:

```
try {
    FileWriter writer = new FileWriter("filename.txt");

    writer.write("Text to save");

    writer.close();

    System.out.println("Data saved successfully.");
} catch (IOException e) {
    System.out.println("An error occurred.");
}
```

The statement below creates a file named `filename.txt`:

```
FileWriter writer = new FileWriter("filename.txt");
```

The `write()` method places text inside the file:

```
writer.write("Text to save");
```

The `close()` method finishes the writing process and closes the file:

```
writer.close();
```

The try block contains the file-writing statements.

The catch block runs when Java encounters a problem while creating or writing to the file.

Let's Practice

Create a Java program named `PersonalProfileFile`.

The program must:

- Ask the user to enter their full name.
- Ask the user to enter their age.
- Ask the user to enter their favorite subject.
- Display the information clearly on the screen.
- Save the information in a file named `personal_profile.txt`.
- Display a message confirming that the file was saved.

The text file should follow this format:

```
PERSONAL PROFILE  
Name: Juan Dela Cruz  
Age: 17  
Favorite Subject: Computer Programming
```

Let's Reflect

Answer the following questions:

1. Why is it useful to save program output in a text file?
2. What is the purpose of the `write()` method?
3. Why should the program close the `FileWriter` after writing?
4. What may happen if the program does not use try and catch?
5. How did Lessons 21, 22, and 23 work together in the challenge?